

Learnlytics

handbook

Mathematics & Statistics For DS



Concise



Comprehensive



Career-Defining



LEARN SMART.
ANALYZE BETTER.
GROW FASTER.

MATHEMATICS FOR DATA SCIENCE

PART 02

A Complete 13-Chapter Study Guide

First Edition

Published by Learnlytics.handbook

www.learnlyticshandbook.shop | learnlytic.handbook@gmail.com

Copyright © 2026 Learnlytics.handbook. All rights reserved.

No part of this publication may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means — electronic, mechanical, photocopying, recording, or otherwise — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

For permission requests or licensing inquiries, write to the publisher at the address above.

ISBN: 978-1-2565-5895-9 (Digital Edition)

First published: 2026

Printed and distributed digitally worldwide.

IMPORTANT: LICENSE & COPYRIGHT NOTICE This document and all included materials are the exclusive intellectual property of Learnlytics.handbook. Your purchase grants you a single-user, personal license. This file is digitally tracked. Unauthorized reproduction, forwarding, resale, or distribution of this material — whether in private groups, Telegram channels, or public forums — is strictly prohibited and constitutes copyright infringement. Our legal team actively monitors digital distribution networks. Violations of this agreement will result in immediate termination of your access, notification to your educational/professional institution, and formal legal action under applicable international and domestic copyright laws. Thank you for respecting the hard work that went into creating this guide!

Disclaimer: The information in this guide is provided for educational purposes. Mathematical concepts and statistical methods presented here reflect widely accepted academic standards. Always refer to your institution's or organisation's preferred references for domain-specific notation. The author and publisher make no warranty, express or implied, with respect to the material contained herein.

Typeset in Arial | Cover design: Learnlytics.handbook

IN Made in India with  for Data Science Worldwide

TABLE OF CONTENTS

13 Chapters • 104 Topics • A Complete Math for ML Journey

PART 2: Mathematics for Machine Learning

CHAPTER TITLE	PAGE
1 Chapter 1: Vectors, Matrices & Operations	07-30
1.1 What is a Vector 1.2 What is a Matrix	
1.3 Vector Operations 1.4 Matrix Operations	
1.5 Matrix Multiplication 1.6 Transpose	
1.7 Broadcasting in NumPy 1.8 Vectors and Matrices in ML	
2 Chapter 2: Matrix Inverse & Determinant	31-58
2.1 What is the Matrix Inverse 2.2 Computing the Inverse	
2.3 Identity Matrix 2.4 What is the Determinant	
2.5 Computing the Determinant 2.6 Singular Matrices	
2.7 Solving Linear Systems 2.8 OLS as a Matrix Equation	
3 Chapter 3: Eigenvalues & Eigenvectors	59-87
3.1 What are Eigenvectors 3.2 What are Eigenvalues	
3.3 Computing Eigenvalues and Eigenvectors 3.4 Properties of Eigenvalues	
3.5 Symmetric Matrices and Real Eigenvalues 3.6 Diagonalisation	
3.7 Eigenvalues in PCA 3.8 Eigenvalues in ML	
4 Chapter 4: SVD — Singular Value Decomposition	88-117
4.1 What is SVD 4.2 Singular Values	
4.3 Left and Right Singular Vectors 4.4 Computing SVD in NumPy	
4.5 Truncated SVD 4.6 SVD and PCA	
4.7 SVD for Dimensionality Reduction 4.8 SVD in Recommender Systems	
5 Chapter 5: Projections & Orthogonality	118-147
5.1 What is a Projection 5.2 Orthogonal Vectors	
5.3 Orthogonal Matrices 5.4 Gram-Schmidt Orthogonalisation	
5.5 QR Decomposition 5.6 Projection onto a Subspace	
5.7 OLS as Projection 5.8 Orthogonality in ML	
6 Chapter 6: Cosine Similarity	148-178
6.1 What is Cosine Similarity 6.2 Geometric Intuition	
6.3 Cosine Similarity vs. Euclidean Distance 6.4 Computing Cosine Similarity in Python	

[6.5](#) Cosine Similarity for Text [6.6](#) Cosine Similarity for Embeddings
[6.7](#) Cosine Similarity Matrix [6.8](#) Limitations of Cosine Similarity

7 Chapter 7: Gradient Vectors & Hessian **179-210**

[7.1](#) From Scalar Derivative to Gradient Vector [7.2](#) The Gradient as Direction of Steepest Ascent
[7.3](#) Computing Gradients in NumPy [7.4](#) The Jacobian Matrix
[7.5](#) The Hessian Matrix [7.6](#) Hessian and Convexity
[7.7](#) Newton's Method [7.8](#) Gradients in ML

8 Chapter 8: Derivatives & Chain Rule **211-241**

[8.1](#) What is a Derivative [8.2](#) Basic Differentiation Rules
[8.3](#) Common Derivatives in ML [8.4](#) The Chain Rule
[8.5](#) Chain Rule Through Multiple Layers [8.6](#) Symbolic Differentiation with SymPy
[8.7](#) Numerical Differentiation [8.8](#) Derivatives of Loss Functions

9 Chapter 9: Partial Derivatives & Gradient **242-266**

[9.1](#) What is a Partial Derivative [9.2](#) Notation and Computation
[9.3](#) Gradient as Collection of Partial Derivatives [9.4](#) Directional Derivatives
[9.5](#) Computing Partial Derivatives for ML Loss [9.6](#) Autograd in PyTorch for Partial Derivatives
[9.7](#) Gradient of Multi-Output Functions [9.8](#) Partial Derivatives and Feature Sensitivity

10 Chapter 10: Gradient Descent & Variants **267-294**

[10.1](#) What is Gradient Descent [10.2](#) The Learning Rate
[10.3](#) Batch Gradient Descent [10.4](#) Stochastic Gradient Descent (SGD)
[10.5](#) Mini-Batch Gradient Descent [10.6](#) Momentum
[10.7](#) Adaptive Learning Rate Methods [10.8](#) Implementing Gradient Descent from Scratch

11 Chapter 11: Convexity & Loss Functions **295-325**

[11.1](#) What is a Convex Function [11.2](#) Testing for Convexity
[11.3](#) Convex Optimisation [11.4](#) Non-Convex Loss Surfaces
[11.5](#) MSE Loss [11.6](#) Cross-Entropy Loss
[11.7](#) Regularised Loss Functions [11.8](#) Loss Function Selection Guide

12 Chapter 12: Backpropagation Intuition **326-355**

[12.1](#) What Backpropagation Computes [12.2](#) Computation Graphs
[12.3](#) Forward Pass Through a Computation Graph [12.4](#) Backward Pass
[12.5](#) Backprop for a Single Neuron [12.6](#) Backprop for a Two-Layer Network
[12.7](#) Vanishing and Exploding Gradients [12.8](#) How Autograd Implements Backprop

13 Chapter 13: Numerical Optimisation Methods **356-385**

[13.1](#) Why Numerical Optimisation [13.2](#) scipy.optimize Overview
[13.3](#) Gradient-Free Optimisation [13.4](#) L-BFGS
[13.5](#) Constrained Optimisation [13.6](#) Global Optimisation
[13.7](#) Hyperparameter Optimisation as Numerical Optimisation [13.8](#) When to Use Which Method

Chapter 1: Vectors, Matrices & Operations

Introduction

These Handbook cover the linear algebra foundations that sit beneath every tool a data scientist uses daily — from numpy arrays to sklearn models to neural networks. The focus is entirely on intuition, application and code. No proofs. Every formula is decoded in plain English and mapped to numpy.

Each section follows the same structure: Definition → Intuition → Why It Matters → Formula Decoded → Geometric Interpretation → numpy Translation → Connects Forward To → Real-World Analogy → Math Traps. Read linearly or use as a reference — the structure supports both.

1.1 What Is a Vector

► Definition

A vector is an ordered list of numbers. In n-dimensional space it has n components, each representing a coordinate along one axis. Formally: $v = [v_1, v_2, \dots, v_n] \in \mathbb{R}^n$. A single number (scalar) is a 0-dimensional vector; a data row with 10 features is a 10-dimensional vector.

► Intuition First

Think of a vector as a set of GPS instructions: 'go 3 steps right, 4 steps up.' The numbers encode both a direction and a distance in one object. Two vectors that point the same way but differ in length convey the same direction but different magnitude — like the same recipe doubled. A vector lives in space and has an arrowhead.

► Why It Matters for Data Science

Every observation in a dataset is a vector. A customer record [age=25, income=50000, purchases=12] is the vector [25, 50000, 12]. Word embeddings (Word2Vec, GloVe) represent words as dense vectors. Image pixels flatten into a vector. Distance between two feature vectors measures similarity — the entire logic of KNN lives here.

► The Formula — Decoded

```
v = [v1, v2, ..., vn]
```

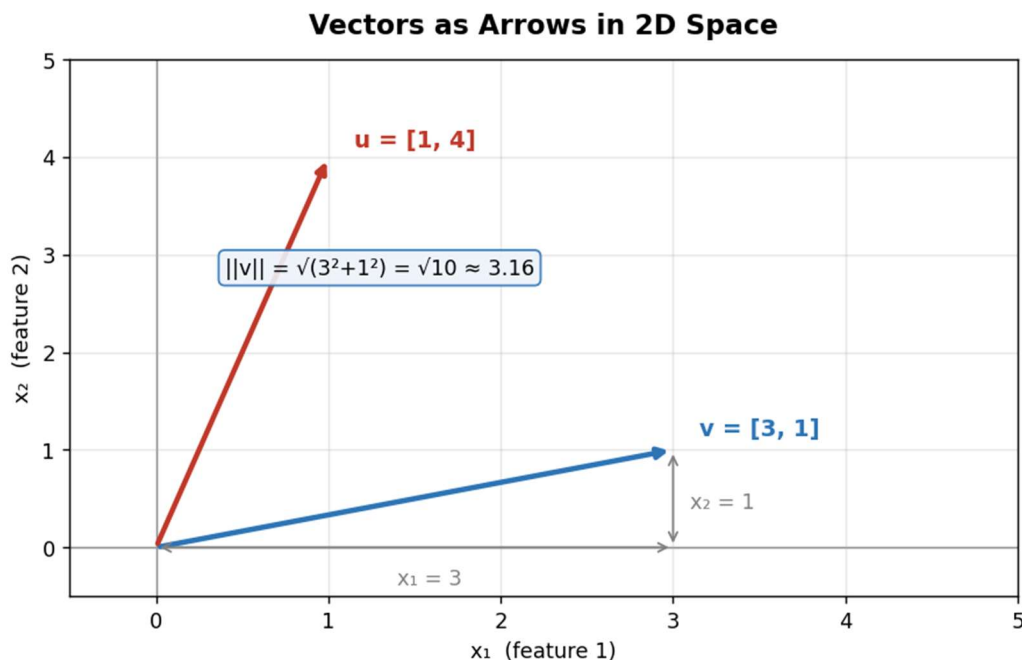
```
Magnitude (length): ||v|| = sqrt(v12 + v22 + ... + vn2)
= np.linalg.norm(v)
```

```
Unit vector (direction only): v̂ = v / ||v|| (magnitude becomes 1)
```

The magnitude is just Pythagoras extended to n dimensions. The unit vector strips out the length and keeps only the direction — dividing a feature vector by its norm is one way to make models direction-sensitive (cosine similarity), not scale-sensitive.

► Geometric Interpretation

In 2D, draw an arrow from the origin (0, 0) to the point (v_1 , v_2). The length of the arrow = the magnitude. The angle it makes with the x-axis = the direction. Two parallel vectors have the same direction; a perpendicular vector pair has a 90° angle between them. The plot below shows two vectors [3,1] and [1,4] in feature space.



► numpy Translation

```
import numpy as np

v = np.array([3, 4]) # shape: (2,) - 1D array
print(v.shape) # (2,)

magnitude = np.linalg.norm(v) # 5.0 (3-4-5 triangle)
unit_v = v / np.linalg.norm(v) # [0.6, 0.8]

# Column vector (2D): needed for matrix operations
col = v.reshape(-1, 1) # shape: (2, 1)
row = v.reshape(1, -1) # shape: (1, 2)
```

► Connects Forward To

- Dot product & cosine similarity (Section 1.3)
- Feature scaling and normalization (affects magnitude)
- Distance metrics — Euclidean distance = $||a - b||$
- KNN, K-means: all distance-based algorithms
- Word embeddings & sentence similarity in NLP

- Neural network input layers (input vector = feature vector)

► Real-World Analogy

[GPS] A delivery address in GPS navigation. 'Go 3 km East, then 4 km North' is the vector [3, 4]. The same instructions always reach the same destination regardless of where you started — vectors describe displacement, not absolute position. The total journey distance is the magnitude: $\sqrt{3^2+4^2} = 5$ km.

► Math Traps for Data Scientists

1. **Shape (3,) ≠ column vector:** `np.array([1,2,3])` is a 1D array of shape (3,). It's not a column vector (3,1). This causes silent shape bugs in matrix multiplication. Always check `.shape` before matrix ops and use `.reshape(-1,1)` when a column vector is needed.
2. **Magnitude vs direction are independent:** Scaling a feature vector [25, 50000, 12] by 2 doubles the magnitude but keeps the direction. KNN uses Euclidean distance (magnitude-sensitive), while cosine similarity uses direction only. Forgetting this means high-valued features silently dominate distance-based models.
3. **Assuming features are on the same scale:** A vector [age=25, salary=80000] has a distance dominated by salary. Always standardize with `StandardScaler` before computing any distances — otherwise the largest-valued feature controls the geometry.

1.2 What Is a Matrix

► Definition

A matrix is a 2D rectangular array of numbers with m rows and n columns, written $A \in \mathbb{R}^{m \times n}$. The element at row i , column j is $A[i,j]$. A vector is a special matrix with exactly 1 column ($m \times 1$) or 1 row ($1 \times n$). A scalar is a 1×1 matrix. Shape is always stated as (rows, columns).

► Intuition First

A matrix is a structured container for multiple vectors — think of it as a spreadsheet where each row is a data point and each column is a feature. But a matrix is also a transformation machine: multiply a matrix by a vector and you move that vector to a new location in space. A 2×2 matrix can rotate, scale, reflect, or shear any 2D vector it touches.

► Why It Matters for Data Science

Your entire training dataset is the matrix X with shape $(n_samples, n_features)$. Every `sklearn.fit(X, y)` receives this matrix. Covariance matrices ($p \times p$) summarise feature relationships for PCA. Neural network weight matrices (`hidden_in × hidden_out`) define layer connections. The normal equation in linear regression involves matrix inversion. Confusion matrices, correlation matrices, distance matrices — all matrices.

► The Formula — Decoded

```
A = | a11 a12 ... a1n | ← row 1 (first data point)
    | a21 a22 ... a2n | ← row 2
    | :: |
    | am1 am2 ... amn | ← row m (last data point)

Shape: (m rows) × (n columns) — always rows × columns
Element access: A[i, j] — row i, column j (0-indexed in numpy)
```

Key shapes to memorize: Dataset = (n_samples, n_features). Weight matrix (single neural layer) = (n_features, n_hidden). Covariance matrix = (n_features, n_features) — always square. Identity matrix I = square with 1s on the diagonal and 0s elsewhere; multiplying any matrix by I leaves it unchanged.

► Geometric Interpretation

In 2D, a 2×2 matrix represents a linear transformation of the plane. The unit square (corners at origin, [1,0], [0,1], [1,1]) gets stretched, rotated, or sheared into a parallelogram. The absolute value of the determinant tells you the area-scaling factor. An identity matrix leaves all vectors unchanged. The matrix [[2,0],[0,2]] doubles every vector's length. The matrix [[0,-1],[1,0]] rotates by 90°.

► numpy Translation

```
import numpy as np

X = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]]) # shape: (3, 3)

print(X.shape) # (3, 3) — 3 rows, 3 columns
print(X[0, 1]) # 2 — row 0, column 1 (0-indexed)
print(X[:, 1]) # [2, 5, 8] — entire second column
print(X[0, :]) # [1, 2, 3] — entire first row

n_samples, n_features = X.shape # always unpack like this

I = np.eye(3) # 3×3 identity matrix
col_means = X.mean(axis=0) # shape (3,) — mean of each column
row_means = X.mean(axis=1) # shape (3,) — mean of each row
```

► Connects Forward To

- Matrix multiplication — the core of every ML forward pass (Section 1.5)
- Transpose — rearranging for dot products and covariance (Section 1.6)
- Linear regression: design matrix X and normal equations
- Covariance matrix $\rightarrow$ PCA $\rightarrow$ dimensionality reduction
- SVD (Singular Value Decomposition) — factorises any matrix
- Neural network weight matrices W between layers

► Real-World Analogy

[Spreadsheet] A school grade book. Rows = students (observations), columns = subjects (features). Each row is one student's complete profile — a vector. The full grade book is the data matrix. Computing the class average per subject is `X.mean(axis=0)` — averaging down each column. Sorting students by their total score is `np.argsort(X.sum(axis=1))`.

► Math Traps for Data Scientists

4. **Row-first indexing:** $A[i, j]$ is row i , column j — always. numpy prints matrices row by row. Forgetting this causes you to slice the wrong dimension. Make it a habit: 'first index is the row.'
5. **Never skip .shape:** Before any matrix operation print `X.shape`. A shape mismatch is the #1 source of cryptic numpy errors (`ValueError: operands could not be broadcast`). Checking shape takes one line and saves hours.
6. **1D arrays are not matrices:** `np.array([1,2,3])` has shape `(3,)` — not `(1,3)` or `(3,1)`. numpy treats 1D arrays as neither row nor column vectors; they broadcast ambiguously. Use `.reshape(1,-1)` for a row matrix or `.reshape(-1,1)` for a column matrix when interacting with 2D functions.

1.3 Vector Operations

► Definition

Three core operations: (1) Addition — add corresponding elements pairwise. (2) Scalar multiplication — multiply every element by a single number. (3) Dot product — multiply corresponding elements then sum them all. These operations are the building blocks of every linear algebra computation in ML.

► Intuition First

Vector addition combines two movements into one final displacement — like walking north then east: the result is the single diagonal step that gets you there. Scalar multiplication stretches or shrinks a vector along its direction without turning it. The dot product measures the degree of

'agreement' between two vectors — how much do they point in the same direction? Two completely perpendicular vectors always give a dot product of zero.

► Why It Matters for Data Science

The dot product is the single most important operation in ML. Linear regression prediction: $\hat{y} = w \cdot x$ (weights dot-producted with features). Cosine similarity (used in recommendation systems, NLP) is a normalised dot product. The entire forward pass of a neural network is a sequence of dot products. Attention scores in transformers are dot products between query and key vectors.

► The Formula — Decoded

Addition: $a + b = [a_1+b_1, a_2+b_2, \dots, a_n+b_n]$

Scalar multiply: $c \cdot a = [c \cdot a_1, c \cdot a_2, \dots, c \cdot a_n]$

Dot product: $a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n$
 $= \sum a_i b_i$ (sum of element-wise products)

Geometric form: $a \cdot b = ||a|| \times ||b|| \times \cos(\theta)$
 where θ = angle between the two vectors

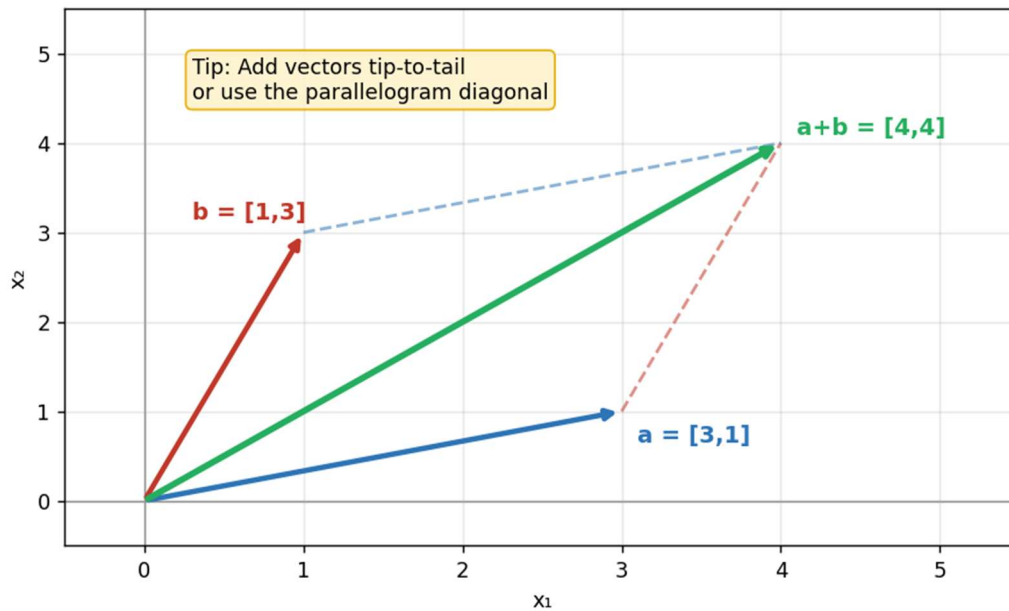
Cosine similarity: $\text{sim}(a,b) = (a \cdot b) / (||a|| \times ||b||) \in [-1, 1]$

The geometric form is the key insight: when $\theta = 0^\circ$ (same direction), $\cos(0)=1$ so $a \cdot b = ||a|| \cdot ||b||$ (maximum). When $\theta = 90^\circ$ (perpendicular), $\cos(90^\circ)=0$ so $a \cdot b = 0$. When $\theta = 180^\circ$ (opposite directions), $a \cdot b$ is negative. This is why cosine similarity of -1 means opposite, 0 means unrelated, and 1 means identical in direction.

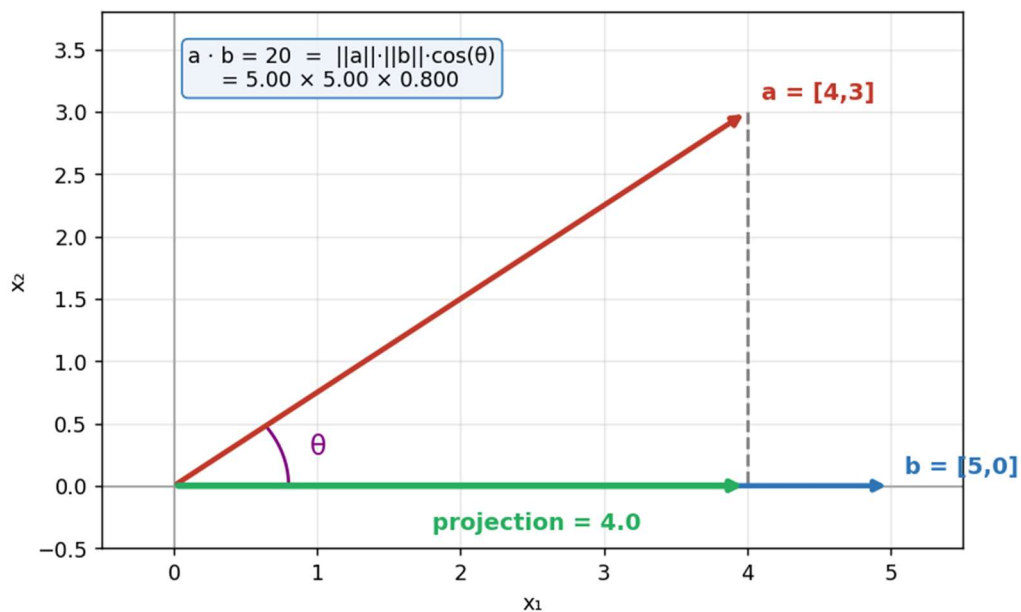
► Geometric Interpretation

Vector addition follows the parallelogram rule: place vectors tip-to-tail and draw the diagonal of the resulting parallelogram — that diagonal is the sum. For the dot product, imagine shining a light perpendicular to vector b ; the shadow cast by vector a onto b has length $(a \cdot b) / ||b||$. The longer the shadow, the more the two vectors 'agree'.

Vector Addition — The Parallelogram Rule



Dot Product — The Shadow / Projection Interpretation



► numpy Translation

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

add = a + b # [5, 7, 9] — element-wise
scaled = 2.5 * a # [2.5, 5.0, 7.5]
```

```
dot = np.dot(a, b) # 32 (= 1×4 + 2×5 + 3×6)
dot = a @ b # same: 32 (preferred modern syntax)

# Cosine similarity (manual):
cos_sim = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)) # 0.9746

# sklearn version:
from sklearn.metrics.pairwise import cosine_similarity
cos_sim = cosine_similarity(a.reshape(1,-1), b.reshape(1,-1))[0,0]
```

► Connects Forward To

- Cosine similarity → text search, recommendation engines, NLP
- Linear regression prediction: $\hat{y} = X @ w$ (Section 1.5 & 1.8)
- Attention mechanism in transformers: $\text{score} = Q @ K^T$
- Kernel methods and SVMs (kernel = dot product in feature space)
- Gradient descent: gradients involve dot products

► Real-World Analogy

[Voting] A voting scorecard. You have a vector of your priorities (weights): [economy=0.5, healthcare=0.3, environment=0.2]. A politician has a policy alignment vector: [economy=0.8, healthcare=0.6, environment=0.4]. Their alignment score = dot product = $0.5 \times 0.8 + 0.3 \times 0.6 + 0.2 \times 0.4 = 0.66$. A politician who perfectly matches your priorities in every dimension would score maximum. This is exactly how a recommendation engine scores items against a user preference vector.

► Math Traps for Data Scientists

- 7. Element-wise (*) vs dot product (@):** $a * b$ computes $[a_1b_1, a_2b_2, a_3b_3]$ — a vector of the same size (Hadamard product). $\text{np.dot}(a, b)$ / $a @ b$ produces a single scalar. These are fundamentally different operations and numpy will not warn you if you use the wrong one.
- 8. np.dot() is NOT cosine similarity:** The dot product scales with magnitude. Two identical vectors $[10,0]$ and $[10,0]$ have dot product 100, while $[1,0]$ and $[1,0]$ have dot product 1. For similarity you must normalize: divide by both norms. Always use sklearn's `cosine_similarity` or normalise manually.
- 9. Dot product shape rules:** For 1D arrays $a @ b$ works. For 2D matrices, the rule is $(m,k) @ (k,n) \rightarrow (m,n)$. If you try $(3,4) @ (3,4)$ numpy throws a `ValueError`. When in doubt: sketch out the shapes before running the code.

1.4 Matrix Operations

► Definition

Matrix operations include: element-wise addition and subtraction (requires identical shapes), element-wise multiplication (Hadamard product, also requires identical shapes), and scalar operations (a single number applied to every element). These are distinct from matrix multiplication (Section 1.5), which has its own rules.

► Intuition First

Element-wise addition is like adding two identically-structured spreadsheets cell by cell. If Matrix A is January sales and Matrix B is February sales, $A + B$ gives the two-month totals — cell by cell, no interaction between different cells. Scalar operations are even simpler: multiplying every element by 0.9 applies a 10% discount uniformly to every entry.

► Why It Matters for Data Science

Feature standardization — the most critical preprocessing step in ML — is entirely matrix element-wise operations: $X_scaled = (X - \text{mean}) / \text{std}$. Subtracting the mean is matrix subtraction via broadcasting. Dividing by std is scalar/vector division. Regularization adds a penalty matrix element-wise to the weight matrix. Gradient updates in training: $W = W - \text{learning_rate} * \text{gradient}$ — element-wise subtraction.

► The Formula — Decoded

```
Element-wise add: (A + B)[i,j] = A[i,j] + B[i,j] — requires same shape
Element-wise sub: (A - B)[i,j] = A[i,j] - B[i,j]
Hadamard product: (A ⊙ B)[i,j] = A[i,j] × B[i,j] — element multiply
Scalar multiply: (c × A)[i,j] = c × A[i,j] — c is a number
```

Feature standardization (Z-score):

```
X_scaled = (X - μ) / σ
```

```
μ = column means (shape: n_features)
```

```
σ = column std (shape: n_features)
```

```
→ broadcasting handles the shape mismatch automatically
```

Standardization transforms each feature to have mean 0 and standard deviation 1. After standardization, a value of +2 means '2 standard deviations above the mean' — the same interpretation for every feature regardless of its original scale. This is what StandardScaler does internally.

► Geometric Interpretation

Adding a constant to every element of a matrix shifts all data points uniformly in one direction — like translating a scatter plot. Multiplying every element by a positive scalar stretches the data uniformly from the origin — like zooming in. Neither operation changes the shape of the data

distribution (the cloud of points), only its position and scale. Standardization first shifts the cloud to center on the origin, then scales it to unit variance.

► numpy Translation

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

add = A + B # [[6,8],[10,12]] - element-wise
sub = A - B # [[-4,-4],[-4,-4]]
elem_mult = A * B # [[5,12],[21,32]] - Hadamard (NOT matrix mult)
scalar = 3 * A # [[3,6],[9,12]]

# Feature standardization (manually):
X = np.random.randn(100, 5)
mean = X.mean(axis=0) # shape (5,) - mean of each feature
std = X.std(axis=0) # shape (5,)
X_scaled = (X - mean) / std # broadcasting: (100,5) - (5,) → (100,5)

# Gradient descent weight update:
W = W - learning_rate * gradient # element-wise subtraction
```

► Connects Forward To

- Broadcasting — how numpy handles mismatched shapes (Section 1.7)
- Feature normalization — StandardScaler is $(X - \text{mean}) / \text{std}$
- Batch normalization in neural networks
- Gradient descent: every weight update is element-wise subtraction
- Residuals in regression: $y_{\text{true}} - y_{\text{pred}}$ (element-wise)

► Real-World Analogy

[Menu] A pizza chain adjusting its price menu. The current prices are a matrix (items × regions). Adding a flat £1.00 to every item is scalar addition. Increasing each item's price by a different percentage is element-wise multiplication by a rate matrix. The key rule: you can only combine two menus element-wise if they have the exact same items and regions — same shape requirement.

► Math Traps for Data Scientists

10. **A * B is NOT matrix multiplication:** In numpy, the * operator does element-wise Hadamard multiplication — not matrix multiplication. Both matrices must have the same shape for * to work. For matrix multiplication use `A @ B` or `np.dot(A, B)`. This is the single most common matrix mistake in numpy code.
11. **Shape must match exactly for element-wise ops:** `(3,2) + (3,2)` works. `(3,2) + (2,3)` raises a `ValueError` (unless broadcasting applies). Always verify shapes before arithmetic. A shape mismatch that gets silently resolved by broadcasting can produce wrong results.
12. **Integer dtype causes silent overflow:** If `X` has dtype `int8` (values -128 to 127), adding two matrices with values near 100 silently wraps around. Always cast: `X = X.astype(np.float64)` before any arithmetic in a pipeline.

1.5 Matrix Multiplication

► Definition

Matrix multiplication takes matrices `A` (shape `m×k`) and `B` (shape `k×n`) and produces `C` (shape `m×n`). Entry `C[i,j]` is the dot product of row `i` of `A` with column `j` of `B`. The inner dimensions must match: `(m×k) @ (k×n) = (m×n)`. The output shape is the outer dimensions.

► Intuition First

Think of matrix multiplication as 'applying a sequence of transformations at once.' If `A` transforms data from space 1 to space 2, and `B` transforms from space 2 to space 3, then `A @ B` transforms directly from space 1 to space 3. Each output element is a vote: how much does row `i` of `A` 'agree with' column `j` of `B`? That vote is a dot product.

► Why It Matters for Data Science

Linear regression: $\hat{y} = X @ w$ — one matrix multiplication gives predictions for all samples simultaneously. Neural network forward pass: $h = \text{activation}(X @ W + b)$ — matrix multiplication through every layer. PCA projection: $X_{\text{reduced}} = X @ \text{components.T}$. Every speed advantage in deep learning comes from GPUs that are optimised specifically to perform matrix multiplication in parallel.

► The Formula — Decoded

$$C = A @ B$$

$$C[i, j] = \sum_k A[i, k] \times B[k, j] \quad (\text{sum over the shared dimension } k)$$

Shape rule: $(m \times k) @ (k \times n) \rightarrow (m \times n)$

inner dims must match ↑↑

outer dims = output shape

Special cases:

```

(n, p) @ (p,) → (n,) ← matrix × vector: n predictions
(1, n) @ (n, 1) → (1, 1) ← row × column: a scalar (dot product)
(m, 1) @ (1, n) → (m, n) ← outer product: creates a full matrix

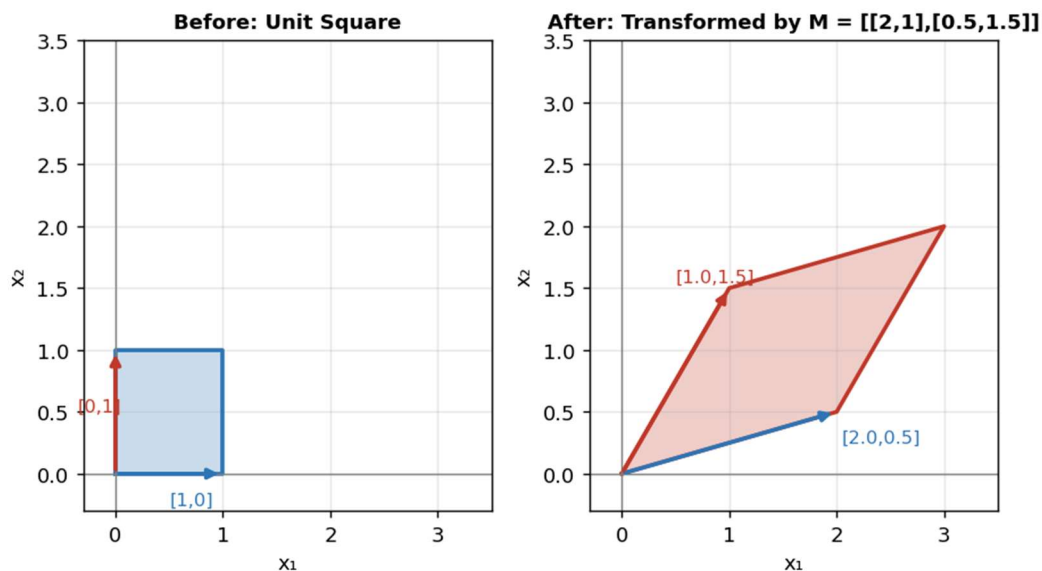
```

Mental shortcut: write the two shapes next to each other — $(3 \times 4)(4 \times 2)$. The two inner numbers must match (both 4). The result takes the two outer numbers: (3×2) . If the inner numbers don't match, the operation is illegal.

► Geometric Interpretation

A 2×2 matrix transforms 2D vectors. The unit square is the 'before' state. After multiplying by the matrix, each corner of the square moves to a new position — the 'after' state. Rotation matrices rotate the square. Scaling matrices stretch it. Shear matrices tilt it. Two matrix multiplications in sequence compose two transformations, producing a single combined transformation matrix.

Matrix Multiplication as a Space Transformation



► numpy Translation

```

import numpy as np

A = np.array([[1, 2],
              [3, 4]]) # shape (2, 2)
B = np.array([[5, 6],
              [7, 8]]) # shape (2, 2)

C = A @ B # [[19,22],[43,50]] — matrix multiplication
C = np.dot(A, B) # identical result

```

```
# Linear regression: predict for all samples at once
X = np.random.randn(100, 5) # 100 samples, 5 features
w = np.random.randn(5) # 5 weights
y_pred = X @ w # shape (100,) - 100 predictions

# Neural network layer
W = np.random.randn(5, 3) # (n_features, n_hidden)
b = np.zeros(3)
h = np.maximum(0, X @ W + b) # shape (100, 3) - ReLU
```

► Connects Forward To

- Linear regression and the normal equation: $w = (X^T X)^{-1} X^T y$
- Neural network forward pass: every layer is a matrix multiplication
- Backpropagation: gradient computation involves matrix products
- PCA projection: $X @ \text{components.T}$ reduces dimensionality
- Attention in transformers: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) \times V$

► Real-World Analogy

[Chain] A currency exchange chain. Matrix A converts USD → EUR (exchange rates for 3 currencies to 2 currencies). Matrix B converts EUR → JPY (2 currencies to 4 currencies). Their product $A @ B$ converts USD directly to JPY — skipping the intermediate step. The crucial constraint: the number of output currencies from A must equal the number of input currencies to B. That's the inner dimension requirement.

► Math Traps for Data Scientists

- 13. Matrix multiplication is NOT commutative:** $A @ B \neq B @ A$ in general. Order matters. In ML, $X @ w$ is different from $w @ X$ (one works, one throws a shape error). Always think about which matrix is on the left.
- 14. Using * instead of @:** $A * B$ performs element-wise (Hadamard) multiplication — both matrices must have the same shape and the result is the same shape. $A @ B$ is true matrix multiplication with the $(m \times k)(k \times n)$ rule. These are completely different computations and numpy won't warn you.
- 15. Shape error on inner dimension:** $(100, 3) @ (100, 2)$ raises `ValueError: shapes not aligned`. The inner dimensions must match: $(100, 3) @ (3, 2)$ works and gives $(100, 2)$. Whenever you get a shape error in @ operations, sketch the shapes on paper and check the middle two numbers.

1.6 Transpose

► Definition

The transpose of a matrix A , written A^T , swaps its rows and columns. If A has shape (m, n) , then A^T has shape (n, m) . The element at position $[i, j]$ in A moves to position $[j, i]$ in A^T . Diagonal elements (where $i = j$) stay in place. Transposing twice gives back the original: $(A^T)^T = A$.

► Intuition First

Transposing is like rotating a matrix 90° and flipping it — rows become columns and columns become rows. If your dataset matrix has patients as rows and measurements as columns, the transpose has measurements as rows and patients as columns. The data hasn't changed, only how it's oriented. For symmetric matrices (like covariance matrices), the transpose is identical to the original.

► Why It Matters for Data Science

The covariance matrix formula is $\Sigma = (1/n) \times X^T X$ — without the transpose, the dimensions don't align. The normal equation for linear regression is $w = (X^T X)^{-1} X^T y$ — transpose appears twice. When computing dot products between column vectors in matrix form you need: $a^T b$ (transpose the left vector). PCA, SVD, and Gram matrices all fundamentally rely on transpose operations.

► The Formula — Decoded

If A has shape (m, n) : A^T has shape (n, m)

Rule: $A^T[i, j] = A[j, i]$

Product transpose rule: $(AB)^T = B^T A^T \leftarrow \text{ORDER REVERSES}$

Sum transpose rule: $(A+B)^T = A^T + B^T$

Symmetric matrix: $A = A^T$ (all covariance matrices are symmetric)

Covariance matrix: $\Sigma = (1/(n-1)) \times X^T X$

X : $(n_samples, n_features)$

X^T : $(n_features, n_samples)$

$X^T X$: $(n_features, n_features)$ — square, symmetric

The rule $(AB)^T = B^T A^T$ (order reverses) is the most commonly forgotten transpose identity. Think of it like putting on shoes and socks: you put on socks first then shoes (A then B), but to reverse you must take off shoes first then socks (B^T then A^T).

► Geometric Interpretation

For a 2D rotation matrix that rotates by angle θ , its transpose rotates by $-\theta$ (the exact reverse). This means the transpose of a rotation is its inverse — a useful property. For the general case, reflecting a matrix about its main diagonal (top-left to bottom-right) gives the transpose.

Symmetric matrices, which equal their own transpose, have a special geometric property: their eigenvectors (principal directions) are always perpendicular to each other.

► numpy Translation

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2, 3)

A_T = A.T # shape (3, 2) - .T is the clean way
A_T = np.transpose(A) # same result

# Covariance matrix:
X = np.random.randn(100, 5) # 100 samples, 5 features
cov = (X.T @ X) / (100 - 1) # (5,100)@(100,5) = (5,5) ✓
# or: np.cov(X.T) # same result

# Normal equation (linear regression):
w = np.linalg.inv(X.T @ X) @ X.T @ y

# WARNING: 1D arrays don't transpose!
v = np.array([1, 2, 3]) # shape (3,)
print(v.T.shape) # still (3,) - .T has no effect on 1D!
col = v.reshape(-1, 1) # shape (3, 1) - proper column vector
print(col.T.shape) # (1, 3) ✓
```

► Connects Forward To

- Covariance matrix $\Sigma = X^T X / (n-1) \rightarrow$ PCA, Mahalanobis distance
- Normal equations in linear regression: $w = (X^T X)^{-1} X^T y$
- Gram matrix: $K = X X^T$ — used in kernel methods and SVMs
- SVD decomposition: $A = U \Sigma V^T$ (three matrices, one transposed)
- Symmetric matrices and eigendecomposition

► Real-World Analogy

[Grade Book] Transposing a spreadsheet in Excel (Paste Special $\rightarrow$ Transpose). A teacher has a grade book where rows = students and columns = subjects. After transposing, rows = subjects and columns = students. The exact same data, just flipped. The diagonal cells

(where student-number equals subject-number) stay in place. The matrix is the same information viewed from a rotated perspective.

► Math Traps for Data Scientists

16. **$(AB)^T = B^T A^T$ — order reverses:** A very common algebra error. $(AB)^T$ is NOT $A^T B^T$. When transposing a product, flip the order. This mistake causes shape mismatches in complex computations like backpropagation.
17. **1D arrays don't transpose in numpy:** `np.array([1,2,3]).T` still has shape (3,). numpy does not change 1D arrays on transpose. You must first make it 2D: `v.reshape(1,-1)` for shape (1,3), or `v.reshape(-1,1)` for shape (3,1). This is perhaps the most surprising numpy behaviour for beginners.
18. **Forgetting transpose in covariance:** `X @ X` computes $(n,p) @ (n,p)$ which requires $n == p$. The correct formula is `X.T @ X` — giving $(p,n) @ (n,p) = (p,p)$. Getting the covariance formula wrong silently produces a meaningless (n,n) matrix instead of the (p,p) feature-covariance matrix.

1.7 Broadcasting in numpy

► Definition

Broadcasting is numpy's automatic mechanism for performing arithmetic between arrays of different shapes. numpy virtually 'stretches' the smaller array along dimensions where its size is 1 (or the dimension is missing), to match the larger array. No data is copied — broadcasting is memory-efficient. If shapes are incompatible, numpy raises a `ValueError`.

► Intuition First

Imagine you have a 1000×5 feature matrix and want to subtract each column's mean. You have 5 mean values, but 1000 rows. Manually you'd need to copy those 5 values into a $(1000,5)$ matrix. Broadcasting does this automatically — it detects that the $(5,)$ array can be 'tiled' 1000 times to match the $(1000,5)$ matrix and applies the subtraction without ever creating that large temporary matrix in memory.

► Why It Matters for Data Science

Every feature normalization step uses broadcasting. Adding bias vectors to neural network layers is broadcasting. Computing pairwise distance matrices for KNN uses broadcasting. Softmax, log-loss, sigmoid — all implemented with broadcasting. Without broadcasting, every operation on a dataset would require Python for-loops, making ML code 10–100x slower.

► The Formula — Decoded

Broadcasting Rules (applied left to right):

Rule 1: If arrays have different ndim, prepend 1s to the smaller shape.

(5,) becomes (1, 5) when broadcast against (3, 5)

Rule 2: Dimensions of size 1 are stretched to match the other array.

(1, 5) stretches to (3, 5) to match (3, 5)

Rule 3: If sizes differ and neither is 1 → ValueError.

Compatible examples:

(1000, 5) - (5,) → (5,) becomes (1, 5) → (1000, 5) ✓

(4, 1) + (1, 3) → result is (4, 3) ✓

(3, 1) * (1, 4) → result is (3, 4) ✓

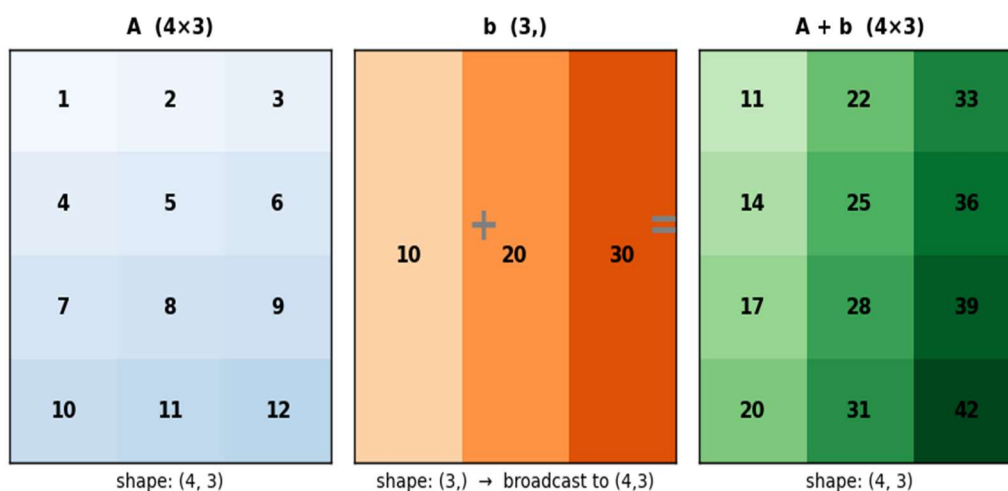
Incompatible example:

(3, 2) + (3,) → (3,) becomes (1, 3) → 2 ≠ 3 → ValueError X

► Geometric Interpretation

Think of broadcasting as tiling a thin slice to fill a volume. A (4,1) column vector broadcast against a (4,3) matrix tiles the column 3 times to fill all columns. A (1,3) row vector broadcast against (4,3) tiles the row 4 times to fill all rows. When both are thin slices — (4,1) + (1,3) — numpy creates the full (4,3) outer-product grid without explicitly allocating it.

Broadcasting: numpy auto-expands (3,) to match (4,3)



► numpy Translation

```
import numpy as np
```

```

X = np.random.randn(100, 5) # dataset: 100 samples, 5 features
mean = X.mean(axis=0) # shape (5,)
std = X.std(axis=0) # shape (5,)

X_scaled = (X - mean) / std # (100,5) - (5,) → (100,5) ✓

# Adding a bias to each sample:
bias = np.array([1, 0, -1, 0, 0.5]) # shape (5,)
X_biased = X + bias # broadcasts to (100,5)

# (4,1) + (1,3) outer-product style broadcast:
col = np.array([[1], [2], [3], [4]]) # shape (4, 1)
row = np.array([[10, 20, 30]]) # shape (1, 3)
result = col + row # shape (4, 3) ✓

# Force a dimension for broadcasting:
v = np.array([1, 2, 3]) # shape (3,)
col_v = v[:, np.newaxis] # shape (3, 1) — now broadcasts differently

```

► Connects Forward To

- Feature normalization — StandardScaler uses broadcasting internally
- Batch normalization in neural networks
- Adding bias vectors in neural network layers ($h = X @ W + b$)
- Vectorised loss functions — no for-loops needed
- Pairwise distance matrices for KNN and clustering

► Real-World Analogy

[Spreadsheet] An Excel formula dragged across cells. You type '=A1 + \$B\$1' in cell C1, then drag it down to C100. Excel automatically applies the same offset from column B to every row — it 'broadcasts' the single cell B1 across all 100 rows. numpy does the same thing in multiple dimensions simultaneously, and does it in optimised C code instead of row-by-row loop logic.

► Math Traps for Data Scientists

19. **Shape (n,) vs (n,1) vs (1,n) behave differently:** np.array([1,2,3]) has shape (3,) and broadcasts along the last axis. np.array([[1],[2],[3]]) has shape (3,1) and broadcasts as a

column. These are not equivalent — $X - v$ vs $X - v.reshape(-1,1)$ produce completely different results when X is 2D.

20. **Silent wrong computation:** If two arrays have accidentally compatible but wrong shapes (e.g. (5,3) and (3,5)), numpy will not raise an error. It broadcasts to (5,5) and produces a meaningless result. Always print the output shape immediately after any broadcasting operation to verify it matches your intent.
21. **Cannot assign into a broadcast result:** You cannot do `a[np.newaxis, :] = 5` to modify `a` through a broadcast view. Broadcasting is read-only. If you need a concrete expanded copy for assignment, use `np.broadcast_to(arr, shape)` or `np.tile(arr, reps)` to create an explicit copy first.

1.8 Vectors and Matrices in ML — The Full Picture

► Definition

In machine learning, every dataset is a matrix X of shape (n_samples, n_features), model parameters are vectors or matrices, and every prediction is computed by a sequence of matrix-vector operations. The entire computational skeleton of ML — from linear regression to deep neural networks — is built on the operations covered in Sections 1.1 through 1.7.

► Intuition First

Machine learning is applied linear algebra with an optimization step. Your data is a matrix. Your model is a set of weight vectors or matrices. Training finds the weights that minimize a loss function — and that loss is always computed using dot products and matrix operations. Even a 100-billion parameter transformer is, structurally, a long chain of matrix multiplications with non-linear functions inserted between layers.

► Why It Matters for Data Science

Understanding the matrix view unlocks your ability to: debug shape errors in model code; write custom transformers; understand why feature scaling affects model convergence; interpret what a weight matrix represents geometrically; reason about computational cost (matrix multiply of $(m,n) \times (n,p) = O(m \cdot n \cdot p)$ operations); and understand why GPU acceleration works (GPUs are designed for parallel matrix multiplication).

► The Formula — Decoded

LINEAR REGRESSION:

$$\hat{y} = X @ w + b$$

X : (n_samples, n_features), w : (n_features,), b : scalar

$\hat{y}$: (n_samples,) — one prediction per row

NORMAL EQUATION (closed-form solution):

$$w^* = (X^T X)^{-1} X^T y$$

NEURAL NETWORK LAYER:

```
h = activation( X @ W + b )
X: (batch, in), W: (in, out), b: (out,)
h: (batch, out) – transformed representation
```

COSINE SIMILARITY (between samples):

```
sim(xi, xj) = (xi · xj) / (||xi|| × ||xj||)
```

PCA PROJECTION:

```
X_reduced = X @ components.T
components: (n_components, n_features) – from PCA.fit()
```

Every line above uses operations from Sections 1.1–1.7. $\hat{y} = X @ w$ is matrix multiplication (1.5). $X^T X$ uses transpose (1.6). $(X - \text{mean}) / \text{std}$ uses broadcasting (1.7). The dot product in cosine similarity is from (1.3). Understanding these sections means understanding what every sklearn model does internally.

► Geometric Interpretation

Each data point is a vector in p -dimensional feature space. The training set is a cloud of n points in that space. Linear regression finds a hyperplane (a $(p-1)$ -dimensional flat subspace) that best fits the cloud — the weight vector w is perpendicular to this hyperplane. PCA finds the directions (eigenvectors) along which the cloud has the greatest spread (variance). A neural network layer applies a matrix multiplication that warps the feature space — folding and stretching it — so that a simple linear boundary can separate the classes.

► numpy Translation

```
import numpy as np
from sklearn.linear_model import LinearRegression

# What sklearn does internally for LinearRegression:
X = np.random.randn(100, 5) # 100 samples, 5 features
y = np.random.randn(100)

# Normal equation:
w = np.linalg.inv(X.T @ X) @ X.T @ y # shape (5,)
y_pred = X @ w # shape (100,)

# More stable: use lstsq
w, _, _, _ = np.linalg.lstsq(X, y, rcond=None)
```

```
# Two-layer neural network (manual):
W1 = np.random.randn(5, 8) * 0.01 # layer 1 weights
b1 = np.zeros(8)
W2 = np.random.randn(8, 1) * 0.01 # layer 2 weights
b2 = np.zeros(1)

h1 = np.maximum(0, X @ W1 + b1) # ReLU: shape (100, 8)
y_pred = h1 @ W2 + b2 # shape (100, 1)

# Cosine similarity between all pairs of rows:
norms = np.linalg.norm(X, axis=1, keepdims=True) # (100,1)
X_norm = X / norms # (100,5) normalised
sim_matrix = X_norm @ X_norm.T # (100,100) cosine sims
```

► Connects Forward To

- Gradient descent: weight update = $W - \text{lr} * \partial L / \partial W$ (matrix subtraction)
- Backpropagation: chain rule expressed entirely in matrix transposes and multiplications
- PCA: eigendecomposition of the covariance matrix $X^T X$
- SVD: factorises any matrix $A = U \Sigma V^T$ — foundation of recommender systems
- Attention mechanism: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$
- All of Part 3 (Probability & Statistics) connects through the covariance matrix

► Real-World Analogy

[Bill] A restaurant bill calculation. Ten friends dine together. Each friend ordered different quantities of dishes — this is the matrix X where rows = friends and columns = dishes. The prices per dish are the weight vector w . Each friend's bill is one row of X dotted with w . All 10 bills at once = $X @ w$ — one matrix multiplication. Training a model is finding the price vector w such that the predicted bills best match the actual totals. That's linear regression.

► Math Traps for Data Scientists

22. **Not scaling features before distance-based models:** A feature matrix with [age, salary, children] = [30, 80000, 2] has salary dominating all distances. KNN, SVM, PCA, and gradient-descent convergence all degrade without StandardScaler. Scaling is not optional — it changes the geometry of the feature space.
23. **Forgetting the bias/intercept term:** $\hat{y} = X @ w$ fits a hyperplane through the origin. Real data rarely passes through the origin. Use `fit_intercept=True` in sklearn (default) or manually add a column of ones to X (the 'bias trick'): `X_aug = np.hstack([X, np.ones((n,1))])`.

24. Using `np.linalg.inv` for the normal equation: `np.linalg.inv(X.T @ X)` is numerically unstable when features are correlated or when `n_features > n_samples`. Use `np.linalg.lstsq(X, y)` for the closed form or rely on `sklearn` which uses iterative methods (coordinate descent, SGD) that are far more stable for large datasets.

Chapter 1 — Quick Summary Chart

The table below summarises every section on one page. Use it as a rapid reference.

Section	Core Concept	Key numpy Code	Primary Use Case	Most Common Trap
1.1 Vector	Ordered list of numbers with direction & magnitude	<code>np.array([1,2,3])</code> <code>np.linalg.norm(v)</code>	Feature representation, similarity, embeddings	Shape (3,) ≠ column vector. Use <code>.reshape(-1,1)</code> for column vectors
1.2 Matrix	2D grid: dataset as (samples × features) or transformation	<code>np.array([[...]])</code> <code>X.shape</code>	Dataset X, weight matrices, covariance matrix	Forgetting to check <code>.shape</code> before ops — source of most errors
1.3 Vector Ops	Addition, scalar multiply, dot product ($\sum a_i b_i$)	<code>np.dot(a,b)</code> <code>a @ b</code> <code>np.linalg.norm(a)</code>	Weighted sums, cosine similarity, attention scores	<code>a * b</code> is element-wise. <code>np.dot(a,b)</code> is the dot product (scalar)
1.4 Matrix Ops	Element-wise arithmetic on same-shaped matrices	<code>A + B</code> , <code>A - B</code> <code>A * B</code> (Hadamard) <code>c * A</code>	Feature scaling, residuals, gradient updates	<code>A * B</code> is Hadamard (element-wise) — NOT matrix multiplication
1.5 Matrix Mult	$(m \times k) @ (k \times n) \rightarrow (m \times n)$: row-col dot products	<code>A @ B</code> <code>np.dot(A, B)</code>	Neural net forward pass, regression prediction, PCA projection	Not commutative. Inner dims must match. <code>*</code> ≠ <code>@</code>
1.6 Transpose	Swap rows ↔ columns: $(m,n) \rightarrow (n,m)$	<code>A.T</code> <code>np.transpose(A)</code>	Covariance $X^T X$, normal equations, SVD	$(AB)^T = B^T A^T$ (order reverses). 1D <code>.T</code> has no effect
1.7 Broadcasting	Auto-expand arrays for arithmetic across dimensions	<code>(100,5) - (5,)</code> <code>v[:, np.newaxis]</code>	Standardization, bias addition, vectorised losses	<code>(5,)</code> vs <code>(5,1)</code> vs <code>(1,5)</code> behave very differently
1.8 ML View	ML = matrix ops on feature matrices: $\hat{y} = Xw + b$	<code>y = X @ w + b</code> <code>X.T @ X</code> (cov)	All ML: regression, nets, PCA, similarity search	No feature scaling + forgetting intercept + using <code>inv()</code> on large X